

Contents

IP Exchange Tool — System Architecture Design	1
1. Actors	1
2. High-Level Architecture Diagram	1
3. Component Breakdown	2
4. Data Model (core entities)	3
5. Tech Stack	3
6. Why “modular monolith” instead of microservices	5
7. Data Flow Walkthrough (matches your described process)	5
8. Security & Compliance Notes (India-specific)	5
9. Suggested MVP Cut	5

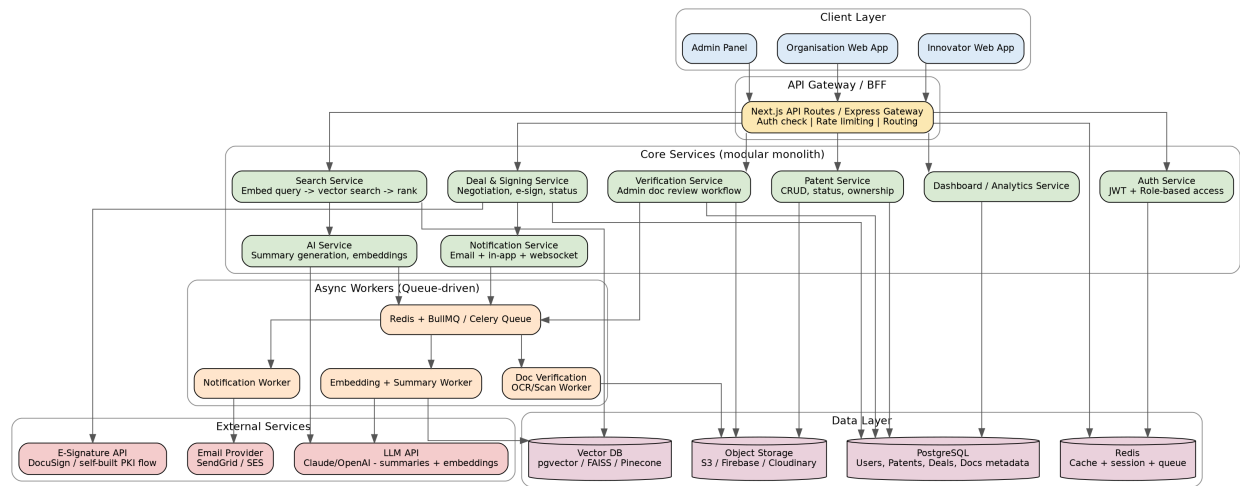
IP Exchange Tool — System Architecture Design

A platform connecting **innovators/creators** (patent sellers) with **organisations** (patent buyers), with an **admin** verifying documents and overseeing deal closure.

1. Actors

Actor	Role
Innovator	Posts patent + proof documents, negotiates, signs, gets paid
Organisation	Searches patents, views AI summaries, negotiates, signs, buys
Admin	Verifies uploaded documents, witnesses/oversees the signing + deal-closure procedure

2. High-Level Architecture Diagram



3. Component Breakdown

3.1 Client Layer

Three surfaces on one codebase (route-guarded by role), or one app with role-based dashboards: - **Innovator app**: profile, patent upload wizard, deal inbox, dashboard (views, interest, sold status) - **Organisation app**: search UI, patent detail view, “request documents” flow, deal tracking - **Admin panel**: verification queue, flagged documents, signing oversight, dispute handling

3.2 API Gateway / BFF

Single entry point that does auth validation, rate limiting, and routes to the right service. In a modular monolith this is just your Next.js/Express app’s route layer; in microservices it’s a real gateway (Kong/NGINX).

3.3 Auth Service

- JWT-based sessions, refresh tokens
- Role-based access control: innovator, organisation, admin
- KYC-lite fields for innovators (since patents + Indian legal docs are involved)

3.4 Patent Service

- CRUD for patent listings: title, abstract, keywords, category, proof documents, status (draft → pending_verification → verified → listed → sold)
- Ownership checks — only the owner or admin can edit

3.5 Verification Service

- Queue of newly submitted patents/documents for admin review
- Checklist against required Indian filing docs (patent application copy, Form 1/3/5 as applicable, assignment deed if any, identity proof)
- Approve/reject with remarks → triggers notification to innovator

3.6 Search Service (the core differentiator)

1. Organisation types a query → query text is embedded (same embedding model used for indexing)
2. Vector similarity search against indexed patent embeddings (title + abstract + keywords)
3. Results ranked by cosine similarity, filtered to status = listed
4. Response returns title + keywords only (full details gated until click, per your flow)

Given you’ve already built a RAG + FAISS pipeline before (CryptoPredictor), the same pattern applies well here: embed on ingestion, store vectors, similarity-search on query.

3.7 AI Service

- On patent verification/listing: generate the short AI summary (LLM call) + generate/store the embedding
- Runs as an **async job**, not inline on the request — summary generation and embedding shouldn’t block the admin’s approve action

3.8 Deal & Signing Service

- State machine: interested → doc_requested → docs_shared → signing → signed → sold
- Every transition logged with timestamp + actor (for audit trail, since admin oversees this)
- Integrates with an e-signature provider, or a simpler self-built flow: both parties upload a signed PDF, admin marks as verified-signed
- On sold: patent status flips, it's excluded from future search results, both dashboards update

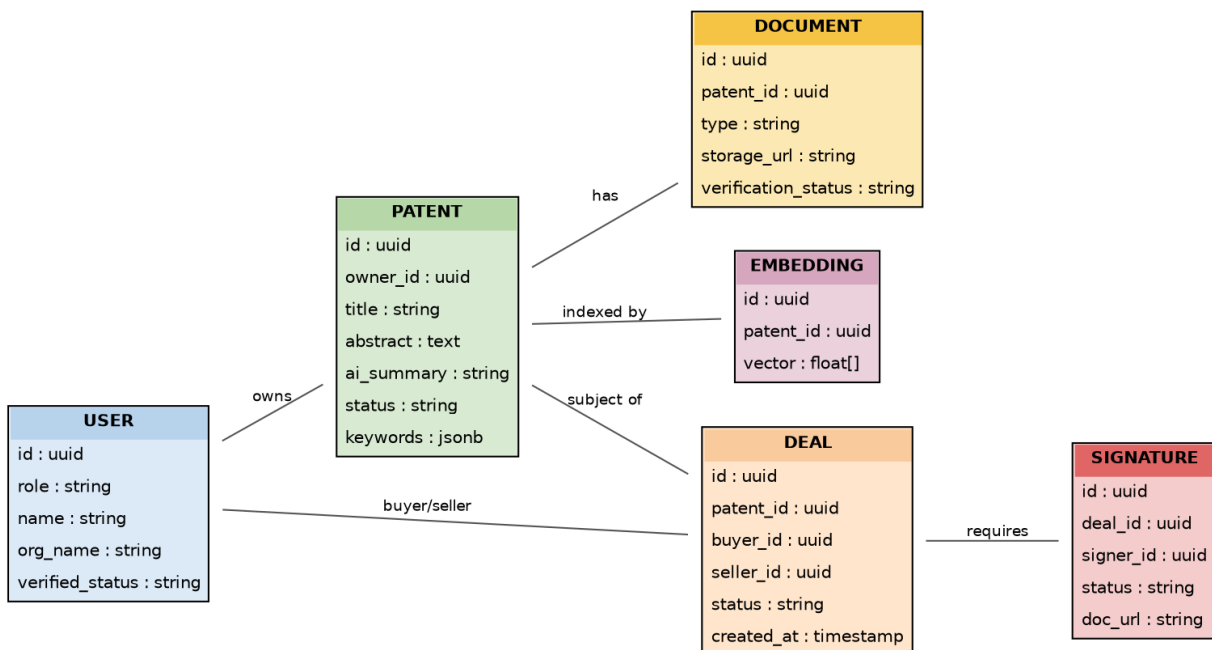
3.9 Notification Service

- Email (deal updates, verification results) + in-app notifications
- WebSocket channel for real-time dashboard updates (e.g., "Organisation X viewed your patent")

3.10 Dashboard/Analytics Service

- Innovator view: views, interest count, active deals, sold patents, earnings (if tracked)
- Organisation view: saved/watched patents, active requests, purchase history
- Admin view: pending verifications, active deals needing oversight, platform-wide stats

4. Data Model (core entities)



5. Tech Stack

Layer	Recommendation	Why
Frontend	Next.js (React) + Tailwind CSS	You already work in this stack (DayForge); SSR helps SEO for public patent listing pages You've used it before (CryptoPredictor); great for wiring in the AI/embedding pipeline directly without a separate service in early stages
Backend API	FastAPI (Python)	
Alternative backend	Node.js/Express, if you want one language across frontend+backend	Simpler for a 2-person team; use langchain.js/direct SDK calls for AI
Auth	JWT + bcrypt, or Firebase Auth / Clerk to save time	Role-based access needed (innovator/org/admin)
Primary DB	PostgreSQL	Relational integrity for users, patents, deals, audit trail
Vector search	pgvector (extension on the same Postgres) to start; FAISS if self-hosted and DB-decoupled; Pinecone/Weaviate if you want managed	You've already built FAISS-based similarity search — reuse that pattern; pgvector avoids running a second DB for a 2-person team
Embeddings + Summaries	OpenAI text-embedding-3-small or Claude for summaries	Keep embedding model fixed once chosen — re-embedding is painful later
File storage	AWS S3 / Firebase Storage / Cloudinary	Patent PDFs, proof documents
Queue/Async	Redis + BullMQ (Node) or Celery (Python)	Decouple slow AI calls and document verification from request/response cycle
Real-time	Socket.io / native WebSockets	Dashboard live updates, "org viewed your patent"
E-signature	DocuSign / SignEasy API, or self-built (upload signed PDF + admin confirms)	Legit e-sign APIs save you from India IT Act compliance headaches around digital signatures
Notifications	SendGrid / AWS SES for email	You already use this
Containerization	Docker (+ docker-compose for local dev)	
CI/CD	GitHub Actions	You already use this
Deployment	Vercel (frontend) + Render/Railway/AWS ECS (backend)	
Monitoring	Sentry (errors) + simple uptime checks	Cheap and fast for an intern-scale project; move to Kubernetes only if load actually demands it Don't over-invest here for MVP

6. Why “modular monolith” instead of microservices

For a 2-person team, running 8 separate microservices adds huge DevOps overhead (service discovery, inter-service auth, distributed tracing) for no real benefit at your current scale. Build it as **one backend codebase with clearly separated modules** (auth/, patents/, verification/, search/, ai/, deals/, notifications/) that talk to each other via function calls, not network calls. This still gives you a clean path to split any module into its own service later — the Search/AI module is the most likely first candidate to split out, since it has different scaling and latency characteristics than the rest.

7. Data Flow Walkthrough (matches your described process)

1. **Post:** Innovator submits patent + docs → Patent Service stores metadata in Postgres, docs in object storage, status = pending_verification → job queued for admin.
 2. **Verify:** Admin reviews in Verification Service → approves → status = verified → async job triggers AI Service to generate summary + embedding → status = listed.
 3. **Search:** Organisation searches → Search Service embeds query, does vector similarity search against listed patents → returns title + keywords.
 4. **View:** Organisation clicks a result → full patent view with AI summary is served.
 5. **Contact:** Organisation expresses interest → Deal Service creates a Deal record, status = interested → notification to innovator.
 6. **Document exchange:** Innovator shares requested docs (scoped access, not public) → status = docs_shared.
 7. **Signing:** Both parties go through signing flow, admin oversees/confirms → status = signed.
 8. **Close:** Status = sold → patent excluded from future search (filter status != listed) → both dashboards refresh via notification/websocket push.
-

8. Security & Compliance Notes (India-specific)

- **Document confidentiality:** proof documents and patent drafts should be access-scoped (signed URLs with short expiry), not publicly readable object storage links.
 - **Digital signatures:** India recognizes electronic signatures under the **IT Act, 2000** (Section 3A) — using a licensed e-signature provider (DocuSign, Leegality, SignEasy) is safer than building your own PKI for an MVP.
 - **Audit trail:** every state transition (verify, interest, doc share, sign, sold) should be immutably logged — this protects both the platform and the admin’s oversight role if a dispute arises.
 - **PII/Confidentiality:** patent abstracts can be public, but underlying full documents and personal details of the innovator should not be exposed until a deal reaches the docs_shared stage.
-

9. Suggested MVP Cut

If timeline is tight, ship in this order: 1. Auth + Patent CRUD + Admin verification (no AI yet) 2. Basic keyword search (skip vector search initially) 3. Deal request flow (manual document sharing, no e-sign — just status tracking) 4. Add AI summary generation 5. Add vector similarity search 6. Add real e-signature integration 7. Add dashboards/analytics last — they’re a view over data you already have